

LSTM Stock Price Predictor

A full-stack end-to-end ML system that predicts next-day stock closing prices and serves predictions via a REST API & Streamlit dashboard.

450+

Tickers

2

LSTM Layers

60

Day Lookback

7 yrs

Training Data

Python 3.11

PyTorch

FastAPI

Streamlit

yfinance

MLflow

Plotly

scikit-learn

NumPy

Pandas

Table of Contents

01.	Table of Contents
02.	1. Project Overview
03.	2. Tech Stack & Languages
04.	3. Libraries & Dependencies
05.	4. Project Structure
06.	5. The Machine Learning Model
07.	6. Data Pipeline
08.	7. Training Pipeline
09.	8. Prediction Engine
10.	9. REST API (FastAPI)
11.	10. Dashboard (Streamlit)
12.	11. Supported Tickers
13.	12. How the Whole System Works (Flow)
14.	13. Evaluation Metrics
15.	14. Trading Recommendation Logic
16.	15. Configuration Reference
17.	16. How to Run
18.	17. File-by-File Reference
19.	18. Limitations & Disclaimer

? LSTM Stock Price Predictor -- Complete Project Documentation

A full-stack, end-to-end machine learning system that predicts next-day stock closing prices using a stacked LSTM neural network, served via a REST API and a live Streamlit dashboard.

Table of Contents

1. Project Overview
2. Tech Stack & Languages
3. Libraries & Dependencies
4. Project Structure
5. The Machine Learning Model
6. Data Pipeline
7. Training Pipeline
8. Prediction Engine
9. REST API (FastAPI)
10. Dashboard (Streamlit)
11. Supported Tickers
12. How the Whole System Works (Flow)
13. Evaluation Metrics
14. Trading Recommendation Logic
15. Configuration Reference
16. How to Run
17. File-by-File Reference
18. Limitations & Disclaimer

1. Project Overview

This project is an AI-powered stock price predictor that:

- Downloads historical OHLCV (Open, High, Low, Close, Volume) data from Yahoo Finance
- Engineers technical indicators (SMA, EMA, RSI, MACD, Bollinger Bands) as features
- Trains a two-layer stacked LSTM neural network to predict the next day's closing price
- Serves predictions through a FastAPI REST API with confidence intervals
- Provides a live interactive Streamlit web dashboard showing:
 - Next-day price prediction
 - BUY / SELL recommendation
 - Both profit AND loss scenarios side-by-side
 - 30-day historical chart with actual vs predicted prices
 - Full P&L summary (expected return, max upside, max downside)
- Tracks all experiments with MLflow

2. Tech Stack & Languages

Layer	Technology	Language
ML Model	PyTorch (LSTM)	Python
Data Download	yfinance	Python
Data Processing	pandas, numpy	Python
Feature Scaling	scikit-learn (MinMaxScaler)	Python
REST API	FastAPI + Uvicorn	Python
Web Dashboard	Streamlit	Python
Charts	Plotly	Python
Experiment Tracking	MLflow	Python
Model Serialization	PyTorch .pt + joblib .pkl	Python
HTTP Client	httpx	Python
Testing	pytest	Python
Containerization	Docker + docker-compose	YAML / Dockerfile
Configuration	Python dataclass	Python

Primary Language: Python 3.11

3. Libraries & Dependencies

torch==2.12.0	-> Deep learning framework (LSTM model)
yfinance==1.4.1	-> Download stock data from Yahoo Finance
pandas==2.2.1	-> DataFrame operations
numpy==1.26.4	-> Numerical computations
scikit-learn==1.5.0	-> MinMaxScaler for feature normalization
fastapi==0.111.0	-> REST API framework
uvicorn==0.29.0	-> ASGI server for FastAPI
streamlit==1.33.0	-> Web dashboard
httpx==0.27.0	-> Async HTTP client (Streamlit -> API calls)
mlflow==2.12.1	-> Experiment tracking & metric logging
joblib==1.4.0	-> Save/load scaler object
matplotlib==3.8.4	-> Plot charts (actual vs predicted, residuals)
plotly==5.18.0	-> Interactive charts in Streamlit
pytest==8.1.1	-> Unit testing
python-dotenv==1.0.1	-> Environment variable management

4. Project Structure

d:\Stock market\	
+-- src/	<- Core ML source code
+-- __init__.py	
+-- config.py	<- All hyperparameters & paths (single source of truth)
+-- data_pipeline.py	<- Download, feature engineering, scaling, sequencing
+-- model.py	<- LSTMPredictor neural network class
+-- train.py	<- Trainer class (training loop, early stopping)
+-- train_main.py	<- Entry point to run full training pipeline

```
|   +-- predict.py           <- Predictor class (load model, run inference)
```

```
|  +-- evaluate.py          <- Metrics (MAE, RMSE, MAPE, Sharpe ratio), plots
```

|

+++ api/ <- REST API

| +-- __init__.py

```
|   +-- main.py           <- FastAPI app with /predict, /history, /metrics, /health
```

|

| `++ streamlit_app.py` `<- Interactive Streamlit dashboard`

|

+++ models/ <- Saved model artifacts (auto-created after training)

```
|   +-- lstm_model.pt           <- Trained PyTorch model checkpoint
```

```
|  +-- scaler.pkl          <- Fitted MinMaxScaler
```

```
|  +-- metrics.json      <- Last evaluation metrics
```

```
|    +-- plots/                               <- Saved matplotlib charts
```

| `+-+ actual_vs_predicted.png`

| +-- residuals.png

|

+++ data/ <- Raw data cache

| +-+ raw/

|

+++ mlruns/ <- MLflow experiment logs

```
+++ tests/                                <- Pytest unit tests
```



```
+++ requirements.txt      <- Python dependencies
```

```
+- setup.py                                <- Auto-setup script
```

--- Dockerfile <- Docker container definition

```
+++ docker-compose.yml      <- Multi-service container orchestration
```

```
+++ render.yaml                                <- Render.com deployment config
```

```
+++ .env.example                                     <- Environment variable template
```

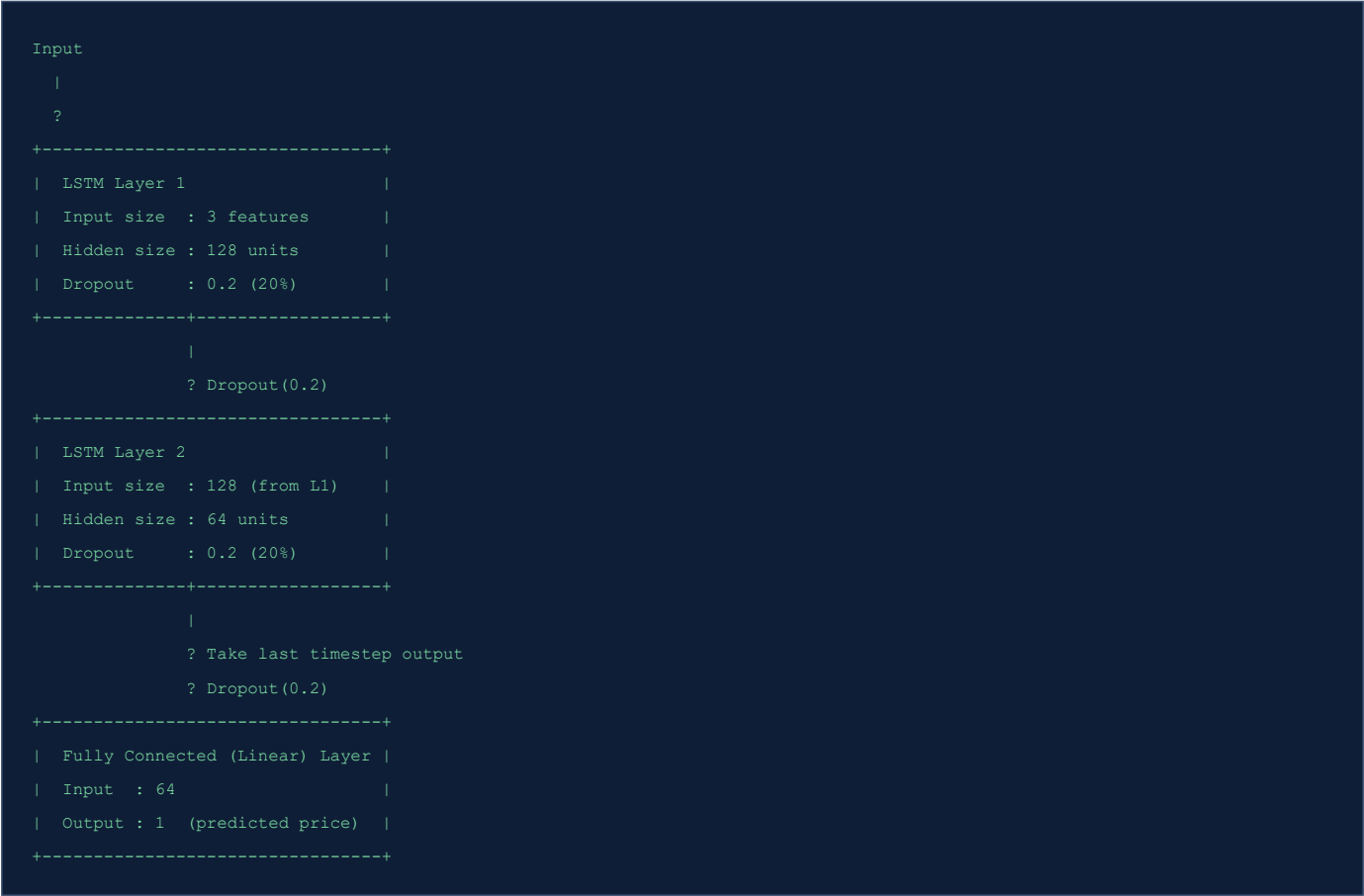
5. The Machine Learning Model

Model Type

Stacked LSTM (Long Short-Term Memory) Neural Network

LSTMs are a type of Recurrent Neural Network (RNN) specifically designed for sequential/time-series data. They can learn long-term dependencies in price sequences -- knowing what happened 60 days ago can influence today's prediction.

Architecture



Model Parameters

Parameter	Value
LSTM Layer 1 Hidden Size	128 units
LSTM Layer 2 Hidden Size	64 units
Dropout Rate	0.2 (20%)
Output Layer	Linear(64 -> 1)
Total Layers	2 LSTM + 1 FC
Input Features	3 (SMA_20, SMA_50, EMA_12)
Sequence Length (Lookback)	60 days

Loss Function

Mean Squared Error (MSE) -- penalizes large price prediction errors quadratically.

Optimizer

Adam (Adaptive Moment Estimation) with learning rate 0.001

Learning Rate Scheduler

ReduceLROnPlateau -- reduces LR by factor 0.5 if validation loss doesn't improve for 5 epochs.

6. Data Pipeline

Data Source

Yahoo Finance via the yfinance library. Supports any ticker symbol available on Yahoo Finance.

Date Range

Default training data: 2018-01-01 to 2024-12-31 (7 years of daily OHLCV data).

Features Used (Input to Model)

The model uses 3 technical indicator features as inputs (not raw OHLCV):

Feature	Description	Window
SMA_20	Simple Moving Average	20 days
SMA_50	Simple Moving Average	50 days
EMA_12	Exponential Moving Average	12 days

Additional Indicators Computed (for display/context, not model input)

Indicator	Description
RSI	Relative Strength Index (14-day)
MACD	Moving Average Convergence Divergence (1
MACD Signal	9-day EMA of MACD
Bollinger Band Lower (BBL)	20-day SMA - 2?
Bollinger Band Upper (BBU)	20-day SMA + 2?

Normalization

All features are scaled to the [0, 1] range using MinMaxScaler from scikit-learn.
The scaler is fit on training data only (no data leakage) and saved to models/scaler.pkl.

Sequence Creation (Sliding Window)

From the scaled time series, the pipeline creates overlapping windows of length 60 days:

- X: 60 consecutive days of features -> shape (N, 60, 3)
- y: The closing price of the next day (day 61) -> shape (N, 1)

Train / Validation / Test Split

Set	Ratio	Purpose
Train	70%	Model learning
Validation	15%	Early stopping, LR scheduling
Test	15%	Final unbiased evaluation

Critical: Split is chronological -- never shuffled. This prevents data leakage in time series.

7. Training Pipeline

Steps (executed by src/train_main.py)

```
[1/5] Download stock data from Yahoo Finance
      v
      Engineer technical indicators
      v
      Normalize with MinMaxScaler (fit on all data)
      v
      Create 60-day sliding window sequences
      v
      Chronological train/val/test split (70/15/15)
      v
      Wrap in PyTorch DataLoaders (batch_size=32)
      v
[2/5] Initialize LSTMPredictor model
      v
      Train with Adam optimizer + ReduceLROnPlateau + Early Stopping
      v
      Save best model checkpoint to models/lstm_model.pt
      v
[3/5] Evaluate on held-out test set
      v
      Compute MAE, RMSE, MAPE, Directional Accuracy, Sharpe Ratio
      v
      Save metrics to models/metrics.json
      v
      Save charts to models/plots/
      v
[4/5] Log all params & metrics to MLflow
      v
[5/5] Print summary
```

Training Hyperparameters

Parameter	Value
Batch Size	32
Max Epochs	100
Learning Rate (initial)	0.001
LR Scheduler Factor	0.5 (halve LR)
LR Scheduler Patience	5 epochs

Early Stopping Patience	10 epochs
Random Seed	42 (fully reproducible)

8. Prediction Engine

Class: `src/predict.py` -> Predictor

Inference Flow (for a given ticker)

```
1. Load saved model weights from models/lstm_model.pt
2. Load saved MinMaxScaler from models/scaler.pkl
3. Download recent historical data for the requested ticker via Yahoo Finance
4. Engineer technical indicators (SMA_20, SMA_50, EMA_12)
5. Scale features using the LOADED scaler (no refitting)
6. Take the last 60 days as the input sequence
7. Run forward pass through the LSTM -> scalar output (scaled)
8. Inverse-transform the output back to real price
9. Compute confidence interval: +/-1.5 x estimated MAE
10. Determine direction: "up" if predicted > current, else "down"
11. Return: predicted_close, confidence_interval, last_actual_close, direction, last_update
```

Historical Prediction (for chart)

The predictor also runs inference over the last N days (default 30) to generate a historical comparison of actual vs predicted prices -- used by the Streamlit chart.

9. REST API (FastAPI)

Entry point: `api/main.py`

Base URL: `http://localhost:8000`

Interactive Docs (Swagger): `http://localhost:8000/docs`

Endpoints

GET /health

Returns API and model status.

```
{ "status": "ok", "model": "lstm", "ticker": "AAPL" }
```

JSON

POST /predict?ticker=AAPL&days_ahead=1

Returns next-day price prediction.

```
{
  "ticker": "AAPL",
  "predicted_close": 229.92,
  "confidence_interval": [228.65, 231.19],
  "last_actual_close": 231.44,
  "direction": "down",
```

JSON

"last_update": "2024-12-31",

"generated_at": "2026-06-04T13:55:00"

}

GET /history/{ticker}?lookback_days=30

Returns 30 days of actual vs predicted prices (for chart).

JSON

```
{
  "ticker": "AAPL",
  "dates": ["2024-12-01", "2024-12-02", ...],
  "actual": [232.15, 229.87, ...],
  "predictions": [230.45, 231.10, ...]
}
```

GET /metrics

Returns last evaluation metrics from training.

JSON

```
{
  "MAE": 4.23,
  "RMSE": 6.11,
  "MAPE": 1.85,
  "Directional_Accuracy": 54.2,
  "Sharpe_Ratio": 1.34
}
```

Middleware

- CORS -- allows all origins (suitable for local dev)
- Request Logger -- logs method, path, status code, and response time for every request

Startup

On startup, the API loads the LSTM model and scaler into memory once (global singleton), so every prediction request re-uses the same loaded model without re-loading from disk.

10. Dashboard (Streamlit)

Entry point: app/streamlit_app.py
URL: http://localhost:8501

Features

Section	What It Shows
Sidebar	Ticker input, date range picker, "Run Pr
Key Metrics (5 tiles)	Last Close, Predicted Close, Direction,
Trading Recommendation	Two-column layout -- always shows BOTH p
P&L Summary Bar	Recommendation, Expected P&L/Share, Max
30-Day Chart	Interactive Plotly chart: actual price (
Historical Data Table	Dates, actual prices, predicted prices (
Prediction Summary Table	Full structured table of all prediction
Model Details (expander)	Features list, LSTM architecture, traini

Trading Recommendation Display

The recommendation always shows both sides:

- Left card (BUY scenario):
 - If model says UP -> green success card marked "MODEL RECOMMENDS THIS"
 - If model says DOWN -> blue info card marked "Alternative / Contrarian (HIGH RISK)"
- Right card (SELL/Risk scenario):
 - If model says DOWN -> red error card marked "MODEL RECOMMENDS THIS"
 - If model says UP -> yellow warning card showing downside risk for stop-loss planning

Each card shows: Entry Price, Target/Worst-case Price, Profit or Loss per share, Return %.

11. Supported Tickers

The system supports 450+ ticker aliases across:

?? Indian Markets (NSE suffix .NS)

Category	Examples
Indices	NIFTY, SENSEX, NIFTYBANK, NIFTYIT, NIFTY
Banking	HDFCBANK, ICICIBANK, SBI, KOTAKBANK, AXI
IT	TCS, INFY, WIPRO, HCLTECH, TECHM, LTIM,
Energy	RELIANCE, ONGC, BPCL, NTPC, ADANIGREEN,
Tata Group	TATASTEEL, TATAMOTORS, TATACONSUM, TITAN
Adani Group	ADANIENT, ADANIPORTS, ADANIPOWER, ADANIG
Pharma	SUNPHARMA, DRREDDY, CIPLA, DIVISLAB, LUP
FMCG	HUL, ITC, BRITANNIA, DABUR, MARICO, COLP
Autos	MARUTI, TATAMOTORS, BAJAJ-AUTO, HEROMOTO
Finance/NBFC	BAJFINANCE, BAJAJFINSV, SBICARD, PFC, RE
Real Estate	DLF, LODHA, GODREJPROP, PRESTIGE, BRIGAD
Metals	JSWSTEEL, HINDALCO, VEDL, COALINDIA, SAI

? Global Markets

Category	Examples
US Indices	S&P500 (^GSPC), NASDAQ (^IXIC), DOW (^DJ
Magnificent 7	AAPL, MSFT, GOOGL, AMZN, META, NVDA, TSL
Semiconductors	INTC, AMD, QCOM, MU, TSM, ARM, ASML, LRC
US Financials	JPM, BAC, GS, MS, VISA, MASTERCARD, PYPL
US Healthcare	JNJ, UNH, LLY, ABBV, PFE, AMGN, MRNA, IS
Cloud / SaaS	ORCL, CRM, SNOW, PLTR, PANW, CRWD, DDOG,
European	SHEL, NVO, BP, AZN, GSK, UL, SIEGY, BMW,
Asian	SAMSUNG (005930.KS), TENCENT (0700.HK),
ETFs	SPY, QQQ, VOO, ARKK, IND, XLK, XLF, XLE

Bonds	TLT, AGG, BND, HYG, LQD
Commodities	GLD (Gold), SLV (Silver), USO (Oil), UNG

Ticker Resolution Logic:

- 19. Check alias map (e.g. SBI -> SBIN.NS, AIRTEL -> BHARTIARTL.NS)
- 20. Try ticker as-is
- 21. Try <ticker>.NS (NSE India)
- 22. Try <ticker>.BO (BSE India)

12. How the Whole System Works (Flow)

```
+-----+
|              TRAINING PHASE              |
|      (run once: train_main.py)            |
|                                           |
|  Yahoo Finance --> Data Pipeline --> LSTM Model --> models/ |
|  (2018-2024)      (features,      (train,      lstm_model.pt |
|                   scaling,        validate,    scaler.pkl   |
|                   sequences)      test)        metrics.json  |
+-----+

+-----+
|              INFERENCE PHASE              |
|      (live: api + streamlit)              |
|                                           |
|  User Types      Streamlit ---- POST /predict --> FastAPI |
|  Ticker (e.g.    Dashboard      GET /history              |
|  "VOLTAS")       ?               ?                       |
|                  |               Predictor                |
|                  |               loads model              |
|                  |               downloads                |
|                  |               Display:                  |
|                  |               - Predicted Price         |
|                  |               - BUY/SELL cards          |
|                  |               - Profit & Loss           |
|                  |               - 30-day chart            |
+-----+
```

Step-by-step for a live prediction:

- 23. User opens Streamlit at <http://localhost:8501>
- 24. User types a ticker (e.g. RELIANCE) and clicks "Run Prediction"
- 25. Streamlit sends POST http://localhost:8000/predict?ticker=RELIANCE&days_ahead=1
- 26. FastAPI's Predictor downloads RELIANCE.NS data from 2018 to 2024 via yfinance
- 27. Engineers features (SMA_20, SMA_50, EMA_12), scales them
- 28. Takes the last 60 days as input sequence
- 29. Runs forward pass through the 2-layer LSTM
- 30. Inverse-transforms the output -> predicted price (e.g. ?1,299)
- 31. Returns JSON with predicted price, direction, CI, and metadata
- 32. Streamlit also calls GET [/history/RELIANCE?lookback_days=30](http://localhost:8000/history/RELIANCE?lookback_days=30)
- 33. Dashboard renders all cards, charts, tables

13. Evaluation Metrics

Metric	What It Measures
MAE (Mean Absolute Error)	Average absolute dollar error in predict
RMSE (Root Mean Squared Error)	Penalizes large errors more than MAE
MAPE (Mean Absolute Percentage Error)	Error as % of actual price (lower = bett
Directional Accuracy	% of days model correctly predicted UP o
Sharpe Ratio (252-day annualized)	Risk-adjusted return of model's trading

A MAPE < 5% is generally considered good for stock price prediction.

Directional Accuracy > 50% means the model beats random guessing.

All metrics are saved to models/metrics.json and accessible via GET /metrics.

14. Trading Recommendation Logic

PYTHON

```
current_price = last actual closing price
predicted_price = LSTM model output (next day)
price_change = predicted_price - current_price
pnl_pct      = (price_change / current_price) * 100

if direction == "UP" and pnl_pct > 0:
    recommendation = "BUY"
else:
    recommendation = "SELL"
```

The dashboard always shows BOTH profit and loss scenarios:

- Profit scenario = potential gain if buying at current price, selling at predicted
- Loss scenario = potential loss if holding through a predicted DOWN move
- Max Upside = CI upper bound (best case)
- Max Downside = CI lower bound (worst case / stop-loss reference)

15. Configuration Reference

All parameters live in src/config.py (single source of truth):

PYTHON

```
# Data
TICKER      = "AAPL"          # Default training ticker
START_DATE  = "2018-01-01"    # Training data start
END_DATE    = "2024-12-31"    # Training data end
LOOKBACK    = 60              # Days of history per sample

# Features
FEATURES    = ["SMA_20", "SMA_50", "EMA_12"]

# Split
TRAIN_RATIO = 0.70            # 70% training
VAL_RATIO   = 0.15            # 15% validation, 15% test
```


Model

```
HIDDEN_SIZE_1 = 128          # LSTM layer 1 units
```

```
HIDDEN_SIZE_2 = 64          # LSTM layer 2 units
```

DROPOUT = 0.2 # Dropout rate


```
# Training
```

BATCH_SIZE = 32

EPOCHS = 100

LR = 0.001

```
EARLY_STOP_PATIENCE = 10          # Stop if val_loss doesn't improve for 10 epochs
```

```
LR_PATIENCE      = 5          # Reduce LR after 5 epochs without improvement
```

```
LR_FACTOR = 0.5 # Multiply LR by this when reducing
```


Paths

```
MODEL_PATH = "models/lstm_model.pt"
```

```
SCALER_PATH = "models/scaler.pkl"
```

16. How to Run

Prerequisites

Python 3.11

Virtual environment at .venv311/

Step 1 -- Install Dependencies

POWERSHELL

```
cd "d:\Stock market"
python setup.py
# OR manually:
.venv311\Scripts\pip install -r requirements.txt
```

Step 2 -- Train the Model (run once)

POWERSHELL

```
.venv311\Scripts\python.exe src/train_main.py
```

This creates models/lstm_model.pt, models/scaler.pkl, models/metrics.json.

Step 3 -- Start the API (Terminal 1)

POWERSHELL

```
.venv311\Scripts\uvicorn.exe api.main:app --reload
# API available at: http://localhost:8000
# Swagger docs at: http://localhost:8000/docs
```

Step 4 -- Launch Dashboard (Terminal 2)

POWERSHELL

```
.venv311\Scripts\streamlit.exe run app/streamlit_app.py
# Dashboard at: http://localhost:8501
```

Step 5 -- View MLflow (Optional, Terminal 3)

POWERSHELL

```
.venv311\Scripts\mlflow.exe ui
# MLflow at: http://localhost:5000
```

17. File-by-File Reference

File	Role
src/config.py	All hyperparameters, paths, feature name
src/data_pipeline.py	Download, ticker resolution (450+ aliase
src/model.py	LSTMPredictor class: 2-layer LSTM + FC l
src/train.py	Trainer class: training loop, validation
src/train_main.py	Entry point: orchestrates data -> train
src/predict.py	Predictor class: loads model, runs infer

src/evaluate.py	Evaluator class: MAE, RMSE, MAPE, Direct
api/main.py	FastAPI app: /predict, /history/{ticker}
app/streamlit_app.py	Full Streamlit dashboard: calls API, ren
models/lstm_model.pt	Saved PyTorch model weights (created aft
models/scaler.pkl	Saved MinMaxScaler fitted on training da
models/metrics.json	Last evaluation results (MAE, RMSE, MAPE
requirements.txt	All Python package dependencies with pin
setup.py	Auto-setup: creates dirs, installs deps,
Dockerfile	Container definition for the API
docker-compose.yml	Multi-service orchestration (API + dashb
render.yaml	Cloud deployment config for Render.com

18. Limitations & Disclaimer

?? This project is for educational and research purposes only.

It is NOT financial advice. Do NOT make real investment decisions based on model outputs.

Technical Limitations

- Single-step prediction only: Currently predicts only 1 day ahead.
- Training data is static: Model is trained on 2018-2024 data. It does not automatically retrain with new data.
- Features are limited: Only 3 technical indicators (SMA_20, SMA_50, EMA_12) are used as model input. Fundamentals, news sentiment, macroeconomics are not included.
- Confidence interval is approximate: The CI is estimated as +/-1.5 x MAE proxy, not from formal Bayesian or ensemble methods.
- CPU inference: Model runs on CPU only by default. GPU support is built in but not configured.
- Yahoo Finance dependency: Data availability depends on Yahoo Finance API. Delisted or renamed tickers may fail.
- No live retraining: The model does not update itself as new market data arrives.

Why LSTM for Stocks?

LSTMs are well-suited for sequential data and can model temporal dependencies. However, stock markets are influenced by countless external factors that historical prices alone cannot capture. The model learns patterns in past data but cannot predict black swan events, earnings surprises, or geopolitical shocks.